

Strumenti IPC – parte I

UNIX offre vari strumenti per la Comunicazione e Sincronizzazione tra processi. In tale ambito si parla di “**Strumenti IPC - Inter-Process Communication** ” che è possibile classificare in:

- **Data-Transfer Facilities: Pipe, FIFO e Code di Messaggi**
 - **Per comunicare:** un processo scrive i dati e un altro processo legge questi dati.
 - **Contro:** questi strumenti **richiedono 2 trasferimenti** dati fra la memoria utente e quella del kernel: un trasferimento durante la scrittura e un trasferimento durante la lettura.
 - **Memoria Condivisa:**
 - La Memoria Condivisa **permette ai processi di scambiarsi le informazioni mettendole in una regione della memoria condivisa fra i processi.**
 - La Memoria Condivisa **è uno Strumento di Comunicazione Molto Veloce**, poiché la comunicazione non richiede syscall o trasferimenti dati fra la memoria utente e quella kernel.
 - **Contro:** questo vantaggio è bilanciato dalla **necessità di sincronizzare le operazioni** sulla Memoria Condivisa.
 - Esempio: un processo non dovrebbe cercare di accedere a una struttura dati presente nella memoria condivisa mentre un altro processo la sta modificando.
 - **Soluzione:** Il **Semaforo** è lo strumento di sincronizzazione abitualmente utilizzato con la Memoria Condivisa(...)
-

Le *Data-transfer facilities* possono essere ulteriormente suddivise nelle seguenti sottocategorie:

- **Byte stream:** i dati scambiati per mezzo di **Pipe, FIFO** sono uno stream di byte.
 - Ogni operazione di lettura **può leggere un Numero Arbitrario di Byte**, senza considerare la dimensione dei blocchi scritti dallo scrivente.
- **Messaggio:** i dati scambiati con le **Code di Messaggi** hanno la forma di messaggi delimitati.
 - ogni operazione di lettura **Legge un Intero Messaggio**, così come scritto dal processo scrivente.
 - non è possibile leggere parzialmente un messaggio, lasciando il resto sulla IPC, e non è possibile leggere molteplici messaggi con una singola operazione di lettura.

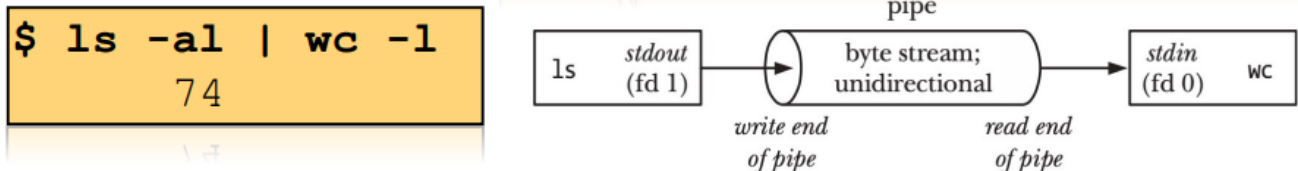
Inoltre, le *Data-transfer facilities* hanno le seguenti caratteristiche fondamentali:

- Sebbene si possano avere molteplici lettori, **Le operazioni di lettura sono distruttive.**
 - Un'operazione di lettura consuma i dati, e i dati non sono più disponibili per altri processi.
- **La sincronizzazione fra processo lettore e scrittore è automatica.**
 - Se un lettore tenta di consumare dati da una facility che attualmente non ne contiene, (di default) l'operazione di lettura si bloccherà finché un processo non avrà scritto dei dati su quella facility.

PIPE

- Flusso di byte uni-direzionale; possiamo vederle come un “tubo”
- PROBLEMA:** Le Pipe permettono la comunicazione tra 2+ processi che sono in relazione di parentela tra di loro
 - Soluzione:** Le **FIFO**; variazione del concetto di Pipe.
 - La differenza sostanziale è che **le FIFO possono essere utilizzate per la comunicazione fra processi qualsiasi**.

Esempio di utilizzo delle PIPE:



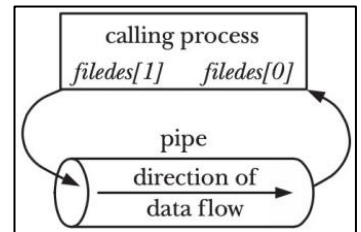
- Per eseguire questi i comandi, la shell crea due processi, che eseguono `ls` e `wc`, rispettivamente; questo è fatto utilizzando (implicitamente) la `fork()` e la `exec()`.
- I due processi sono collegati alla pipe:
 - Il processo scrittore (`ls`) ha il proprio standard out (file descriptor 1) collegato con il *write end of pipe*;
 - mentre il processo lettore (`wc`) ha il proprio standard input (file descriptor 0) collegato al *read end of pipe*.
- NB:** I due processi sono completamente ignari dell'esistenza della Pipe: semplicemente leggono e scrivono da/su descrittori di file standard.

ATTENZIONE:

- Il processo che legge da una Pipe può leggere blocchi di qualsiasi dimensione**, indipendentemente dalla dimensione dei blocchi scritti dal processo che scrive.
- I Byte sono letti nello stesso ordine in cui sono stati scritti;**
 - Non è possibile accedere ai dati in maniera casuale.
- I tentativi di leggere da una Pipe vuota restano bloccati** finché almeno un byte non è stato scritto sulla pipe.
- Le Pipe sono unidirezionali.** I dati possono viaggiare solo in una direzione.
 - Un'estremità (*end*) del pipe è utilizzato in scrittura, e l'altro in lettura.
- Le operazioni di scrittura e lettura sono atomiche** (importante nel caso in cui si avessero molteplici scrittori e lettori): **leggo/scrivo tutti i Byte o nessuno**
- Se il *write end of pipe* viene chiuso (dal processo scrittore), il processo che legge dalla pipe riceverà il codice end-of-file (`read()` restituirà 0) una volta che avrà letto tutti i dati presenti nella pipe.
- Una pipe è semplicemente un Buffer mantenuto in memoria.
 - Questo buffer ha una capacità massima (`PIPE_BUF`; includere `limits.h`).
 - Una volta che una pipe è piena, ulteriori tentativi di scrittura si bloccano** finché il lettore rimuove alcuni dati dalla pipe.
 - In generale, un'applicazione non ha bisogno di conoscere la capacità della pipe
 - Se vogliamo evitare ai processi scrittori di restare bloccati, è necessario che i processi che leggono dalla pipe siano progettati in modo da leggere i dati appena questi sono disponibili
- In teoria, non ci sono motivi per cui una pipe non debba utilizzare capacità minime (es: 1Byte).
 - La ragione per utilizzare Buffer di Dimensioni Maggiori è l'Efficienza:**
 - Ogni volta che uno scrittore riempie la Pipe, il kernel deve eseguire un *context switch* per consentire al lettore di essere schedulato in modo che possa leggere un dato dalla pipe.
 - L'utilizzo di una dimensione maggiore comporta la riduzione del numero di *context switch*.

La syscall `pipe()` crea una nuova Pipe; se va a buon fine (0), la chiamata alloca un array (`filedes`) contenente due descrittori di file aperti:

- Un estremo è aperto in lettura (`filedes[0]`) e l'altro estremo è aperto in scrittura (`filedes[1]`).
- Come con qualsiasi descrittore di file, possiamo utilizzare le syscall `read()` e `write()` per eseguire le operazioni di I/O sulla pipe.



```
#include <unistd.h>

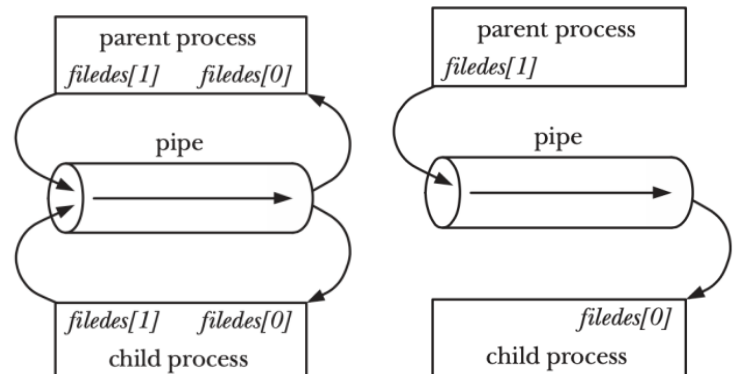
int pipe(int filedes[2]);
//Returns 0 on success, or -1 on error
```

Di norma si utilizzano le Pipe per permettere la comunicazione fra 2 processi:

- Per collegare due processi con una Pipe, eseguiamo prima una system call `pipe()` e poi una `fork()`. Con la `fork()`, il processo figlio eredita copia dei 2 descrittori di file dei genitori.
- Tecnicamente sarebbe possibile leggere e scrivere sulla Pipe per il genitore e il figlio, ma non è il modo standard di utilizzo:
 - subito dopo la `fork()`, un processo chiude il proprio descrittore per l'estremità in scrittura, e l'altro chiude il proprio descrittore per l'estremità in lettura.
 - Per esempio, se il genitore deve inviare dei dati al figlio, deve chiudere l'estremità aperta in lettura, `filedes[0]`, mentre il figlio deve chiudere `filedes[1]`

Questo è necessario per permettere il normale funzionamento della Pipe.

Vedi: “Lettura su Pipe > Dati NON Presenti > File Descriptor Aperto” & “Attenzione: Chiusura dei Descrittori Inutilizzati”.



Lettura su Pipe

Un'operazione di lettura consuma i dati presenti nel buffer della pipe:

- Se i dati sono presenti, la `read` ritorna il numero di Byte letti, che saranno memorizzati nella variabile `buffer`
- Se i dati non sono presenti:
 - Se almeno un file descriptor in scrittura è ancora aperto, allora la `read` è bloccante
 - Se nessun file descriptor in scrittura è aperto, allora la `read` ritorna il valore 0

```
char buffer[100];
int n_bytes;

n_bytes = read(filedes[0], buffer, sizeof(buffer));
```

Scrittura su Pipe

Dopo una operazione di scrittura su una pipe:

- Se il Buffer può contenere i dati passati come parametro: la `write` ritorna il numero di Byte scritti
 - Se nessun file descriptor in lettura è ancora aperto, un segnale di `SIGPIPE` è generato (per notificare che i dati appena scritti non saranno mai letti!)
- Se il buffer della pipe è pieno: la `write` è bloccante (in attesa di una `read`)

```
n_bytes = write(filedes[1], buffer, sizeof(buffer));
```

ATTENZIONE: Chiusura dei Descrittori Inutilizzati

I file descriptors inutilizzati in lettura e in scrittura devono essere chiusi.

- Il processo che legge dalla pipe chiude il proprio *write descriptor*, così che, quando l'altro processo (scrittura) completa il proprio output e chiude il proprio descrittore *write*, il lettore riceve, dopo aver consumato tutti i byte nel buffer, il valore di ritorno 0 (che indica un EOF)
 - Se invece il processo lettore non chiude la propria estremità aperta in scrittura:
 - dopo che l'altro processo (scrittore) avrà chiuso il proprio descrittore *write*, il lettore non riceverà l'end-of-file neppure dopo avere letto tutti i dati dalla pipe.
 - In questo caso, la `read()` si bloccherebbe in attesa di dati (che sappiamo non arriveranno!) perché il kernel sa che c'è ancora almeno un descrittore di file aperto (che è del processo lettore stesso) per la pipe: **DEADLOCK!**
- Il processo che scrive chiude l'estremità aperta in lettura della pipe per una ragione diversa:
 - Quando un processo tenta di scrivere su una Pipe per il quale nessun processo ha un descrittore aperto in lettura, il kernel invia il segnale `SIGPIPE` al processo scrittore.
 - Se un processo intercetta ed ignora tale segnale; in questo caso la `write()` fallisce con un errore `EPIPE` (broken pipe).
 - Ricevere il segnale `SIGPIPE` o ricevere l'errore `EPIPE` è un'indicazione utile in merito allo status della Pipe, ed è la ragione per cui i descrittori aperti in lettura dovrebbero essere chiusi

ATTENZIONE ai Segnali: Se un processo è bloccato in attesa di una lettura o scrittura e viene interrotto da un segnale, l'operazione ritorna -1 con `errno=EINTR`

Scenario Frequente: Comunicazione Tra Processi Nonno-Nipote

- 1) Il "nonno" crea la pipe, e quindi effettua una `fork()`,
- 2) Il "genitore" (figlio del nonno) effettua una ulteriore `fork()` creando così il "nipote".
- 3) Il "nonno" chiuderà uno dei due descrittori della pipe;
- 4) Il "genitore" chiuderà entrambi i descrittori della pipe;
- 5) Il "nipote" chiuderà l'altro descrittore

Scenario Frequente: Comunicazione Tra Processi Fratelli

- 1) Il "genitore" crea la pipe, e quindi effettua una `fork()`,
- 2) Il "genitore" chiuderà entrambi i descrittori della pipe;
- 3) Il "primo fratello" chiuderà uno dei due descrittori della pipe;
- 4) Il "secondo fratello" chiuderà l'altro descrittore

FIFO

Una FIFO è simile a una Pipe. Le FIFO sono anche note come **Pipe con Nome** "named pipes".

- La principale differenza è che **può essere utilizzata per comunicazioni fra processi non imparentati**.
 - Questo è possibile grazie al fatto che ha un nome all'interno del file system ed è aperta nello stesso modo di un file.
- Una volta che una FIFO è stata aperta possiamo utilizzare gli stessi meccanismi visti per le Pipe.
- Possiamo creare una FIFO dalla shell con il comando: `mkfifo [ -m mode ] pathname`
 - Il `pathname` è il nome della FIFO che si intende creare, e
 - l'opzione `-m` specifica i permessi (analogamente al comando `chmod` per i file).
- La funzione `mkfifo()` crea una nuova FIFO con il `pathname` dato.
 - L'argomento `mode` specifica i permessi per la nuova FIFO